

AWS File System

A practical introduction to Amazon Elastic File System (EFS)

Table of Contents

01 Amazon EFS

Shared file storage, mount targets, performance, security and lifecycle management.

02 Practice Lab

Create, mount and share an EFS file system across EC2 instances.

Background

We have established three new company branches. Each branch has its own image server connected to a local client management app. Each server stores client images together with informational metadata, while a custom application attempts to synchronise the data across all three branches.

The problem

Accessing the images takes too much time and the data is not always consistent. The synchronisation process can also fail when one branch server runs out of storage space.

Permissions matter too. For example, folders containing VIP client data may need to be accessible only to the concierge team.

Solution: Amazon EFS

Amazon Elastic File System (EFS) is a serverless shared file storage service that automatically handles capacity and much of the underlying management. Instead of maintaining separate copies at each branch, the servers can access shared files from one central file system.

- Shared network file storage for the client images.
- File-level permissions for restricted folders.
- Storage that grows and shrinks automatically as files are added or removed.

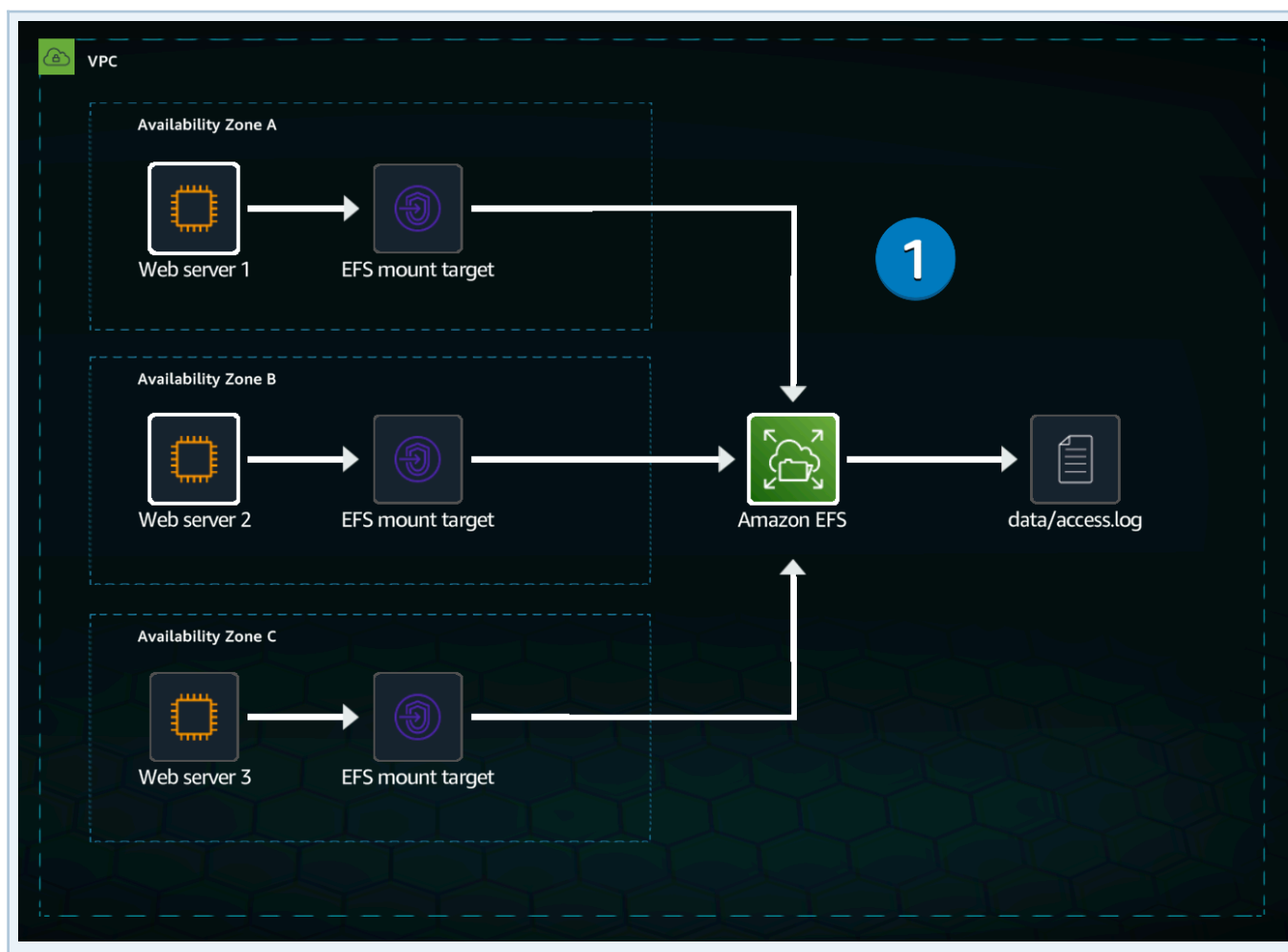
Availability and durability

Amazon EFS is designed to be highly available and highly durable, with Regional file systems storing data across multiple Availability Zones.

01 Amazon EFS

Amazon EFS provides a fully managed, scalable, shared file system that multiple EC2 instances can access at the same time, including instances located across different Availability Zones.

Each Availability Zone can have an EFS mount target, allowing its web servers to access the same central file system. By centralising shared storage for configuration files, images and logs, EFS avoids maintaining separate copies of the same files on each server.



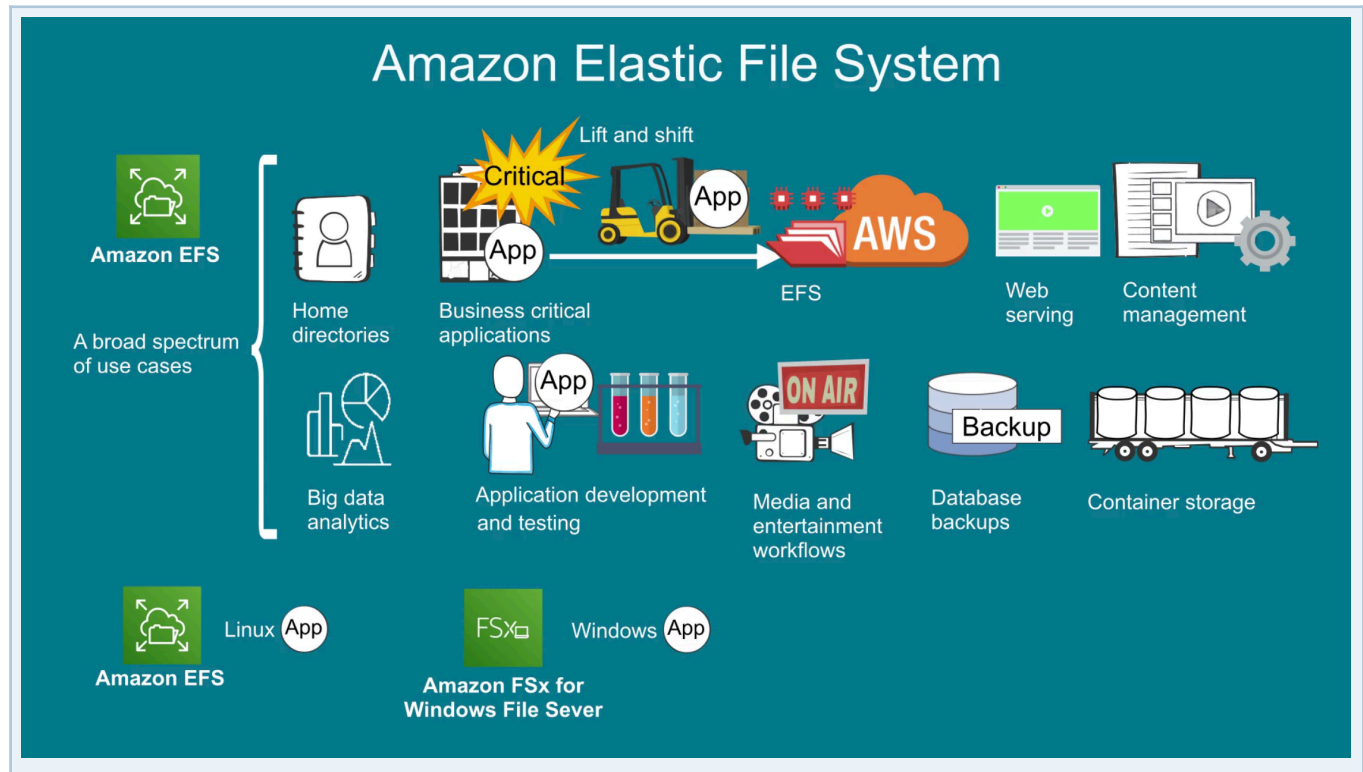
In this example, all three web servers can access the same shared file, such as data/access.log, through Amazon EFS.

Why EFS?

EFS reduces the need to manually deploy, patch and maintain complex shared file-system infrastructure. Existing Linux applications can continue to work with familiar files and directories while EFS scales storage automatically as demand changes.

Amazon EFS is a regional service designed for high availability and durability. On-premises servers can also access EFS through AWS Direct Connect or AWS VPN.

Common use cases include home directories, web serving and content management, application development and testing, media workflows, analytics, database backups, container storage and lift-and-shift migrations.



Amazon EFS is designed for Linux-based applications. For Windows-based applications, AWS provides Amazon FSx for Windows File Server.

Centralised Storage

Amazon EFS allows multiple EC2 instances within a VPC to access the same shared files simultaneously, making it easier to manage files that are frequently updated across multiple servers.

Imagine an online store running on three EC2 web servers. A customer uploads a profile picture while their request is handled by Web Server 1. If the image is stored only on that server, Web Servers 2 and 3 will not have it. With EFS, the image is saved to one shared file system and all three servers can access it.

Why EFS instead of a Database?

A database can store files as binary data, but EFS behaves like a normal shared folder. Applications can work with files using ordinary file operations:

```
/shared/images/product.jpg
```

```
/shared/config/settings.json
```

```
/shared/logs/access.log
```

The operating system sees EFS as a mounted file system, much like another drive or network folder. Existing applications that expect normal files can therefore use it without redesigning file handling around database queries.

The win

EFS gives multiple servers a shared folder that behaves like a normal file system. A database can store the same bytes, but the application must retrieve and manage those bytes as database data.

This is also why lift-and-shift is an important EFS use case. If an existing Linux application expects a path such as `/app/uploads/image.jpg`, EFS can provide that shared filesystem without requiring a major redesign.

Key benefits

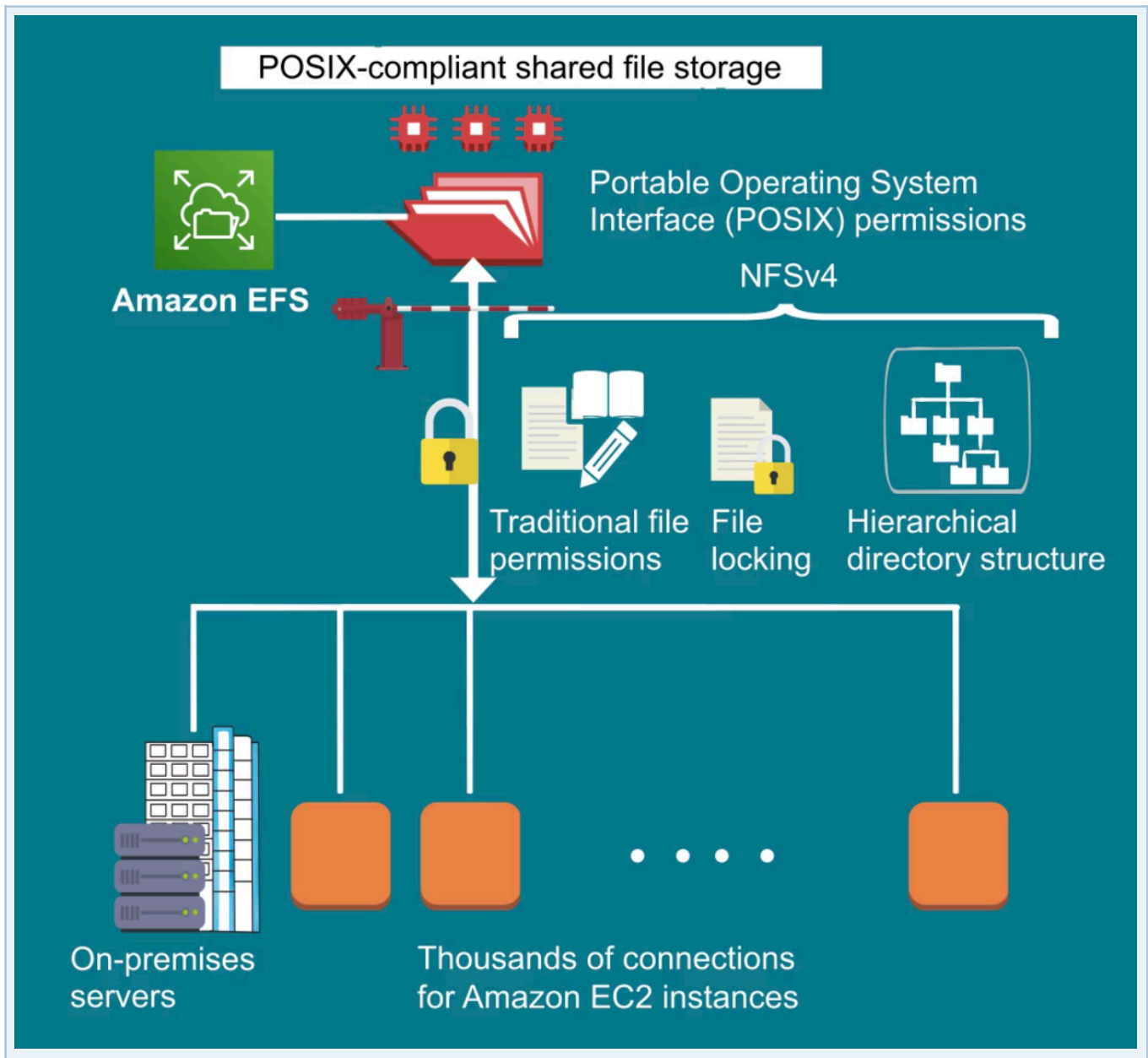
POSIX-compliant file storage

POSIX is a standard associated with Unix/Linux-like operating systems. EFS supports familiar filesystem behaviour such as:

- Read, write and execute permissions.
- File locking to help prevent conflicting access.
- Hierarchical folders and directories.

This allows normal Linux-style permissions, for example: `-rw-r--r-- access.log`

EFS is accessed using NFSv4 (Network File System). A folder such as `/shared` on an EC2 instance can be mounted to EFS, so saving `/shared/photo.jpg` actually stores the file in EFS. A second EC2 instance mounting the same file system can access that same file.

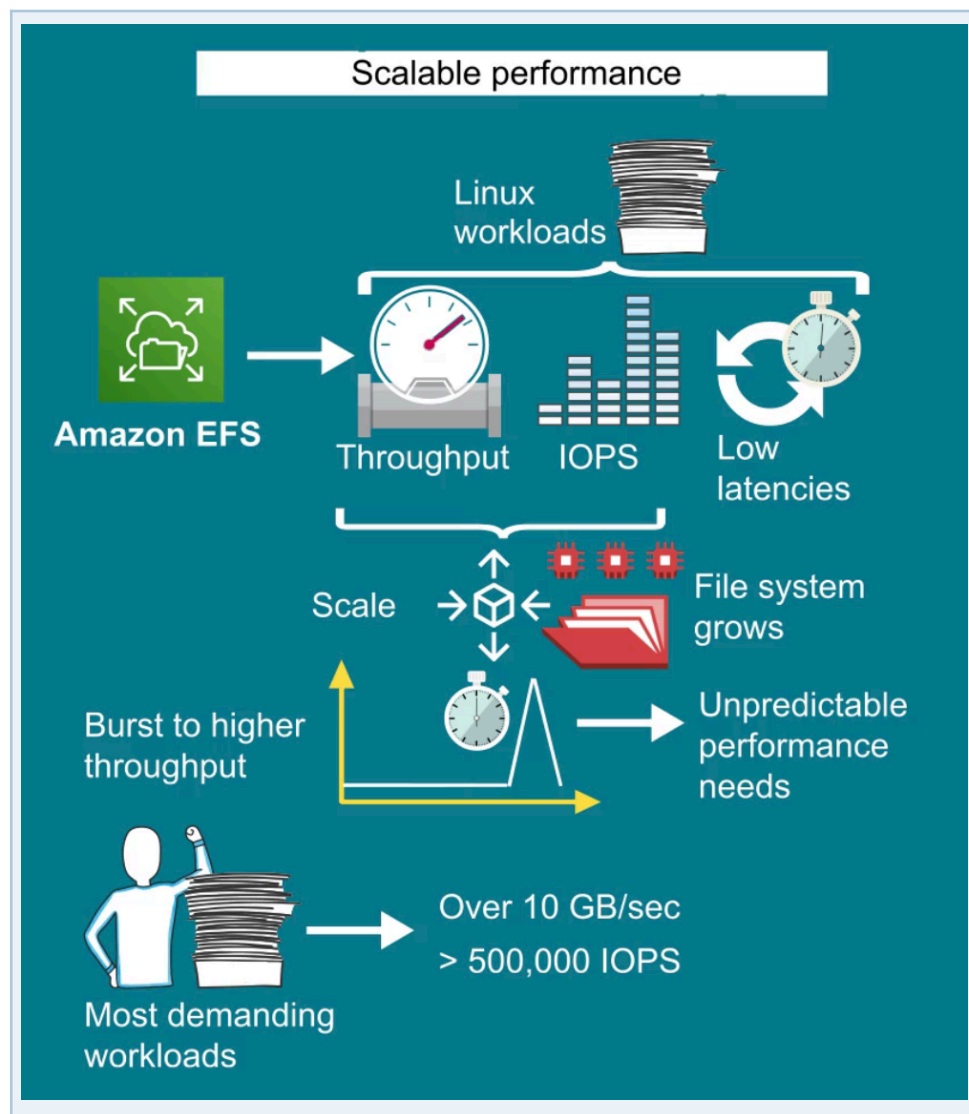


Scalable performance

EFS performance can scale as workloads and the file system grow. As more servers access the file system, EFS can support increasing throughput and file operations while maintaining low latency.

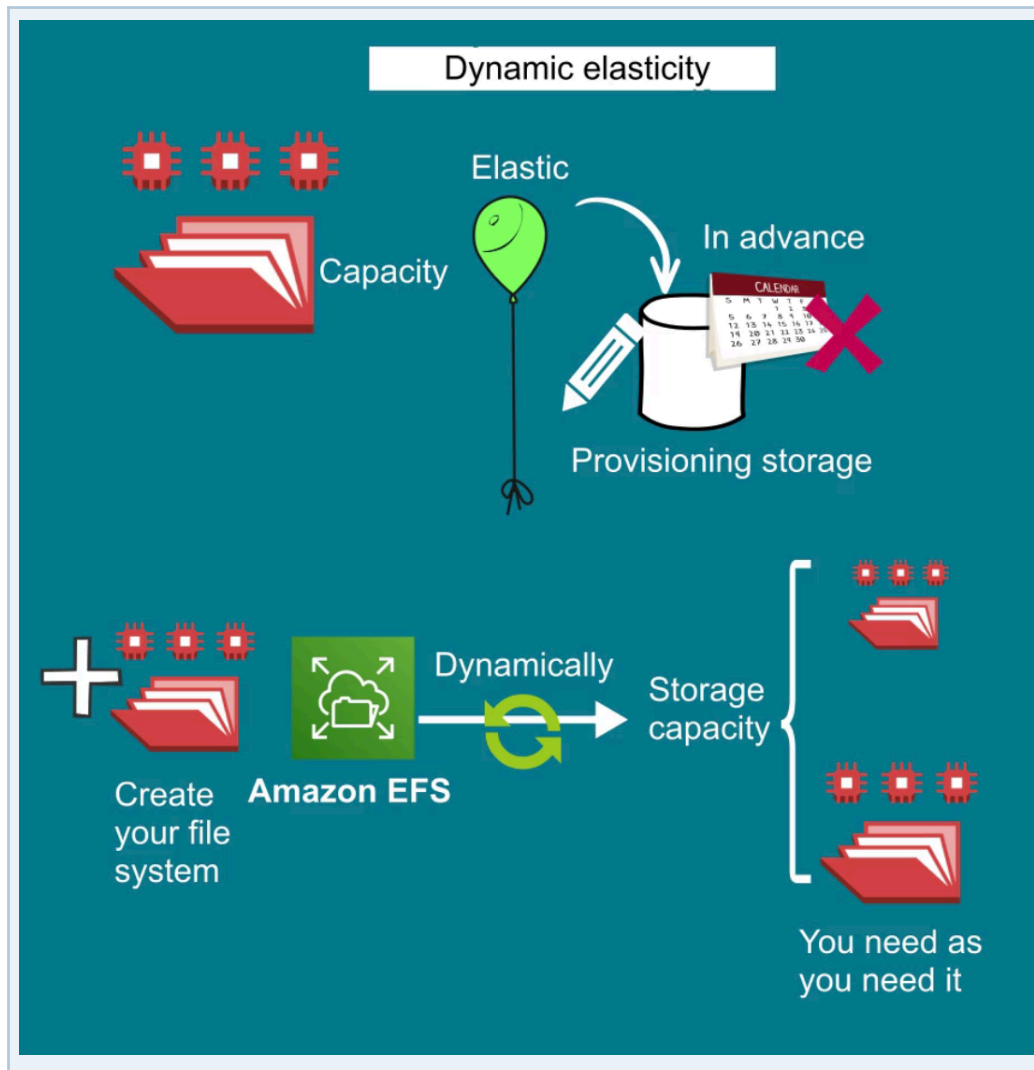
Term	Meaning
Throughput	How much data can be transferred per second.
IOPS	How many read/write operations can be performed per second.
Latency	How quickly an individual operation responds.

EFS can also accommodate bursts and unpredictable changes in demand.



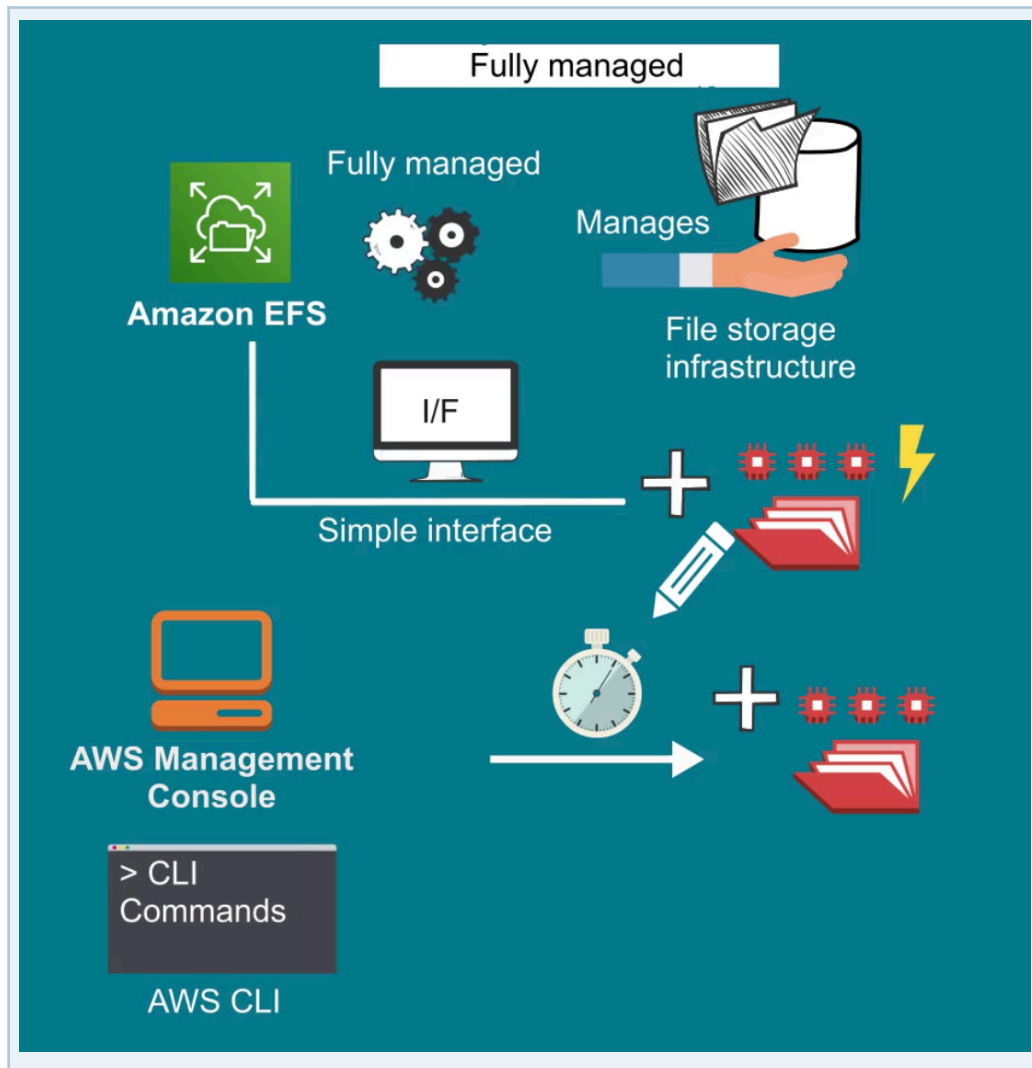
Dynamic Elasticity

EFS automatically grows and shrinks storage capacity as files are added or removed. There is no need to decide in advance how much storage to provision.



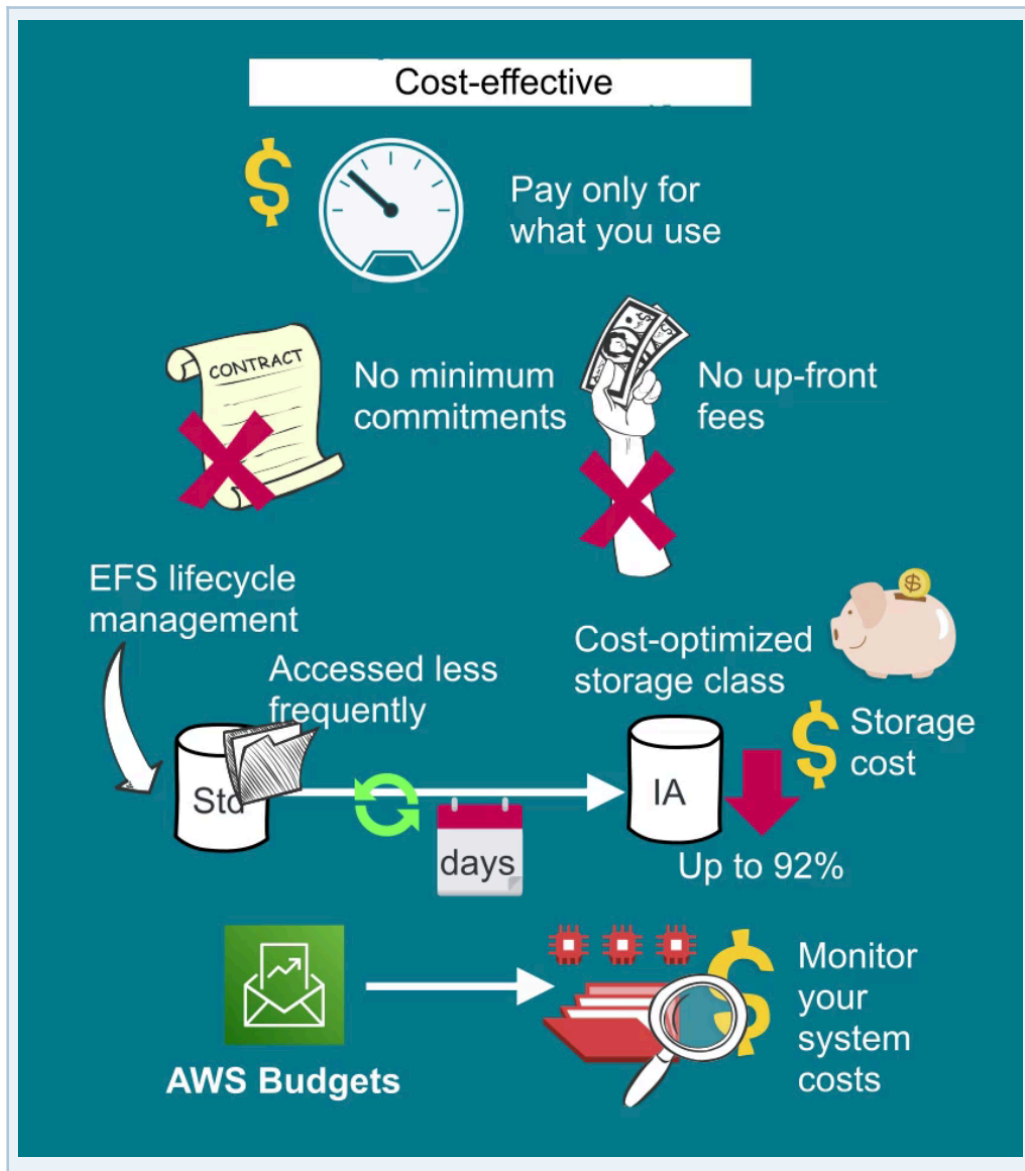
Fully managed

AWS manages the underlying file storage infrastructure. There is no need to provision or maintain storage servers, and EFS can be managed through the AWS Management Console, AWS CLI or AWS SDKs.



Cost-effective

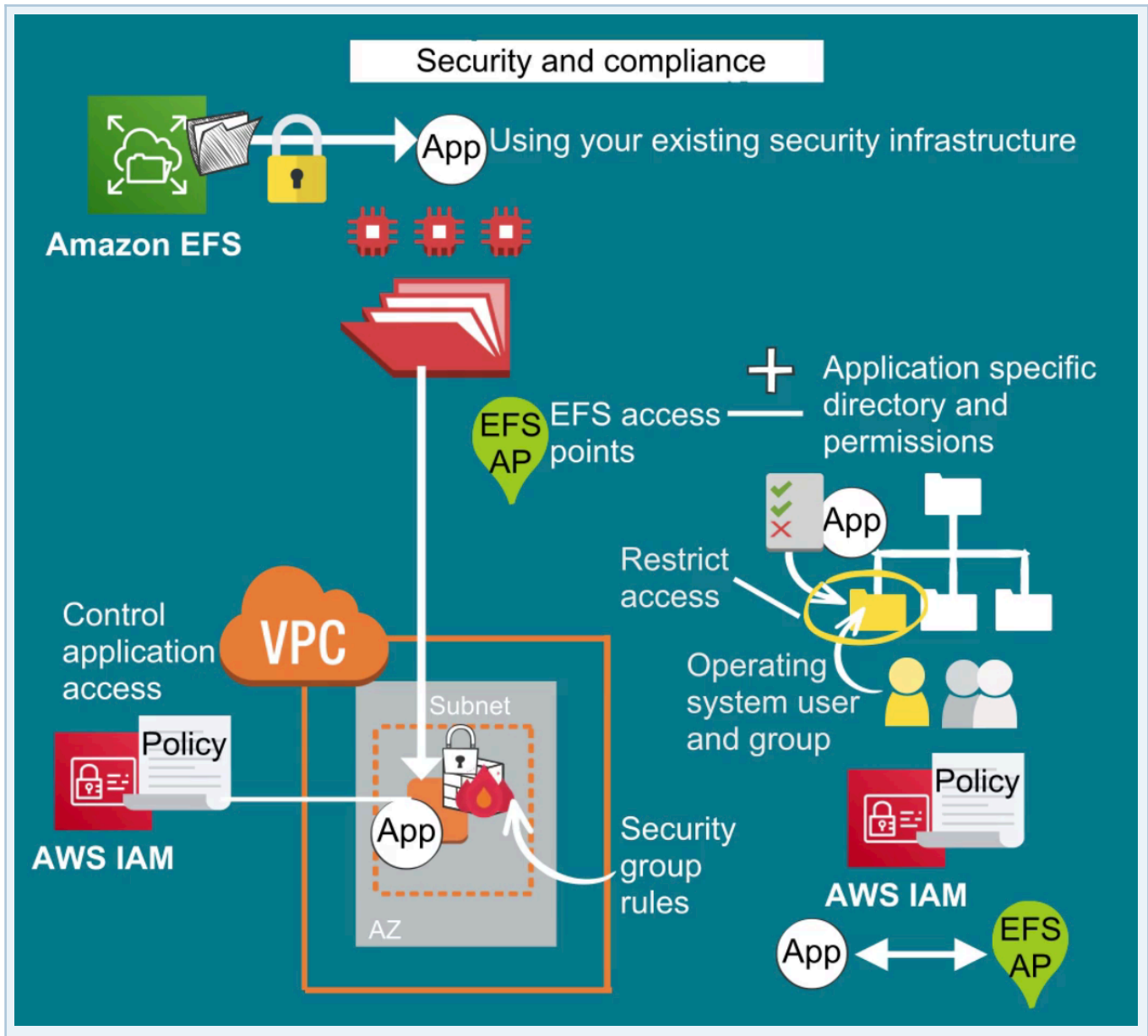
Amazon EFS uses a pay-as-you-go model with no minimum storage commitment or upfront fee. Lifecycle Management can reduce cost further by moving less frequently accessed files into lower-cost storage classes. Management can reduce cost further by moving less frequently accessed files into lower-cost storage classes.



Security and compliance

Amazon EFS uses several layers of AWS and Linux security. Security Groups control network access to EFS within the VPC, while IAM policies can control which identities and applications are authorised to use the file system.

EFS Access Points add application-specific control by directing an application to a particular directory and enforcing Linux user, group and file permissions.



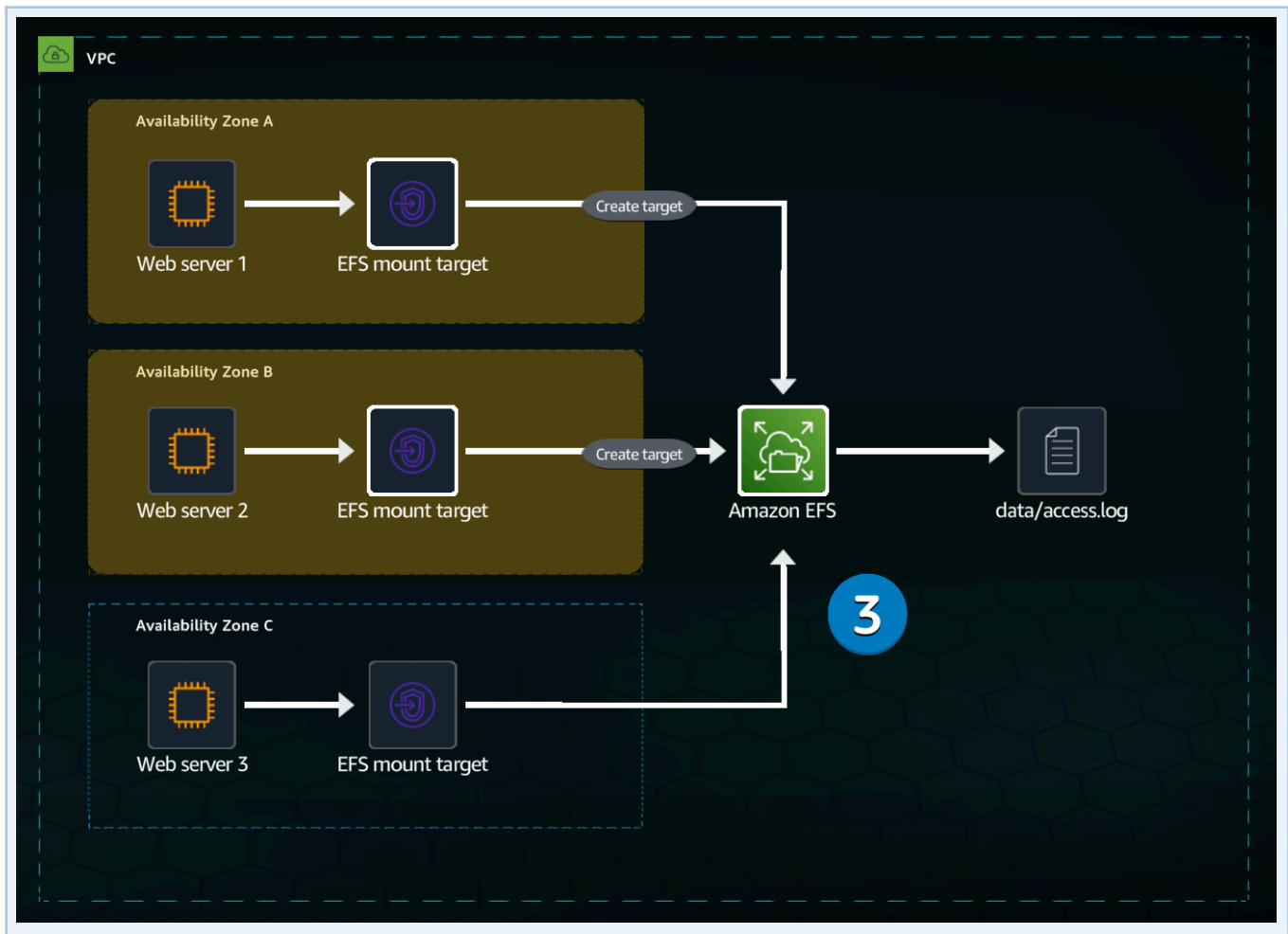
EFS Mount Targets

Amazon EFS is accessed over the network rather than being a disk attached directly to an EC2 instance. A mount target is the network endpoint that provides this connection.

A mount target is created inside a VPC subnet and receives a private IP address that EC2 instances can use to connect to the EFS file system. When EC2 instances are spread across multiple Availability Zones, a mount target can be created in each AZ.

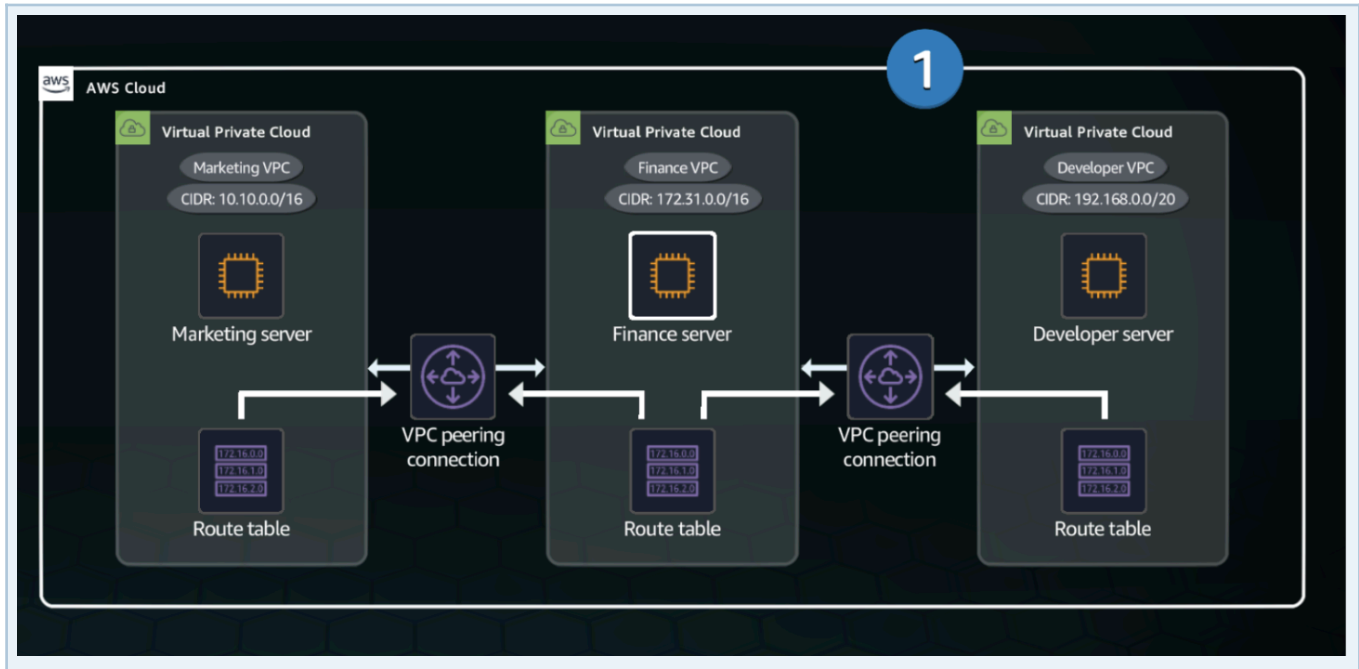
Think of it as a local doorway

There is still one shared EFS file system, but each Availability Zone can have its own network entrance to it.



This probably sounds familiar...

This may sound similar to VPC peering, but the purpose is different. VPC peering connects two separate VPC networks. An EFS mount target is an endpoint inside your VPC that lets resources such as EC2 instances connect to the shared EFS file system.

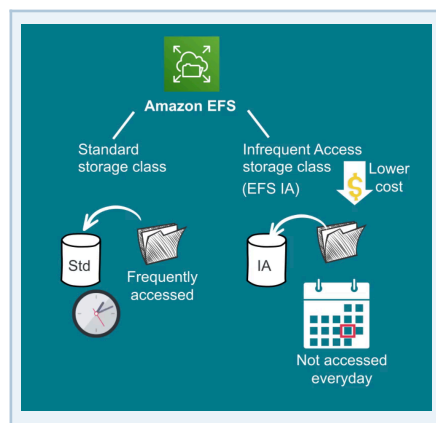


Features

Storage classes

Amazon EFS provides storage classes based on how frequently files are accessed.

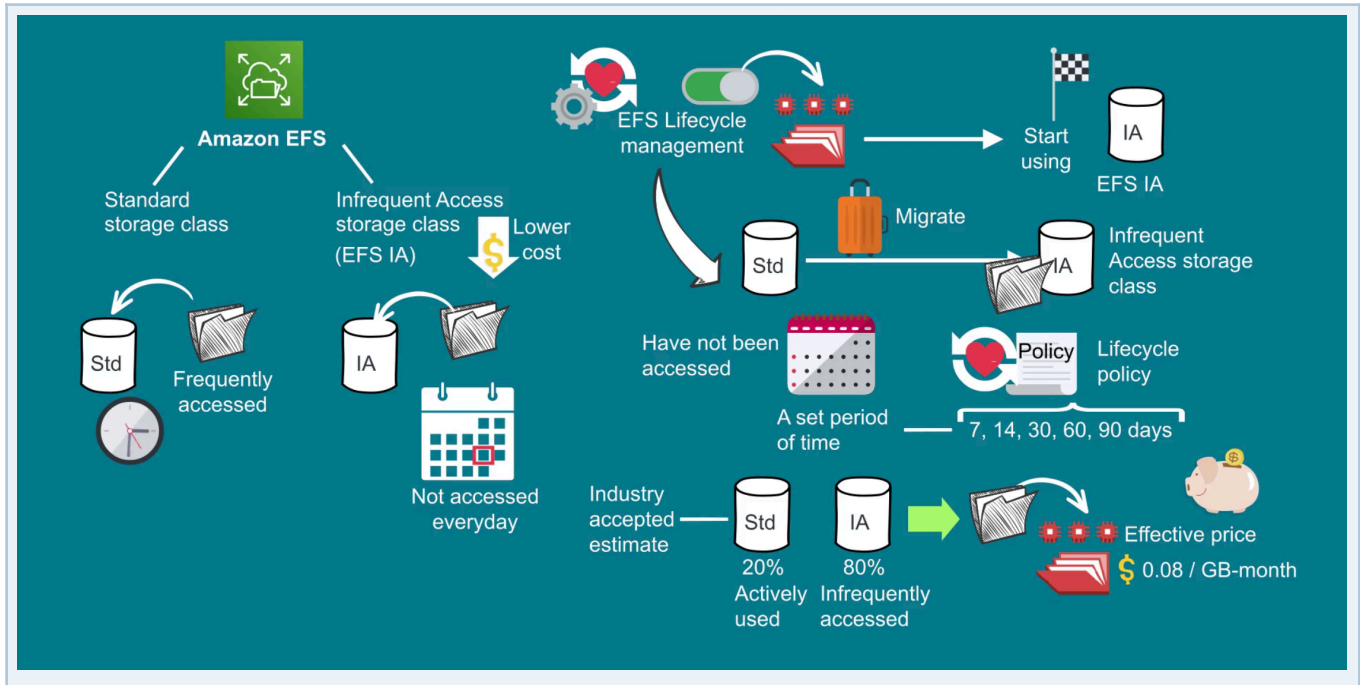
Storage class	Best for
EFS Standard	Files that are accessed frequently.
EFS Infrequent Access (IA)	Files that are not accessed every day.



EFS Lifecycle Management

Lifecycle Management can automatically move files from Standard to lower-cost storage classes after a specified period without access. A lifecycle policy can use periods such as 7, 14, 30, 60 or 90 days.

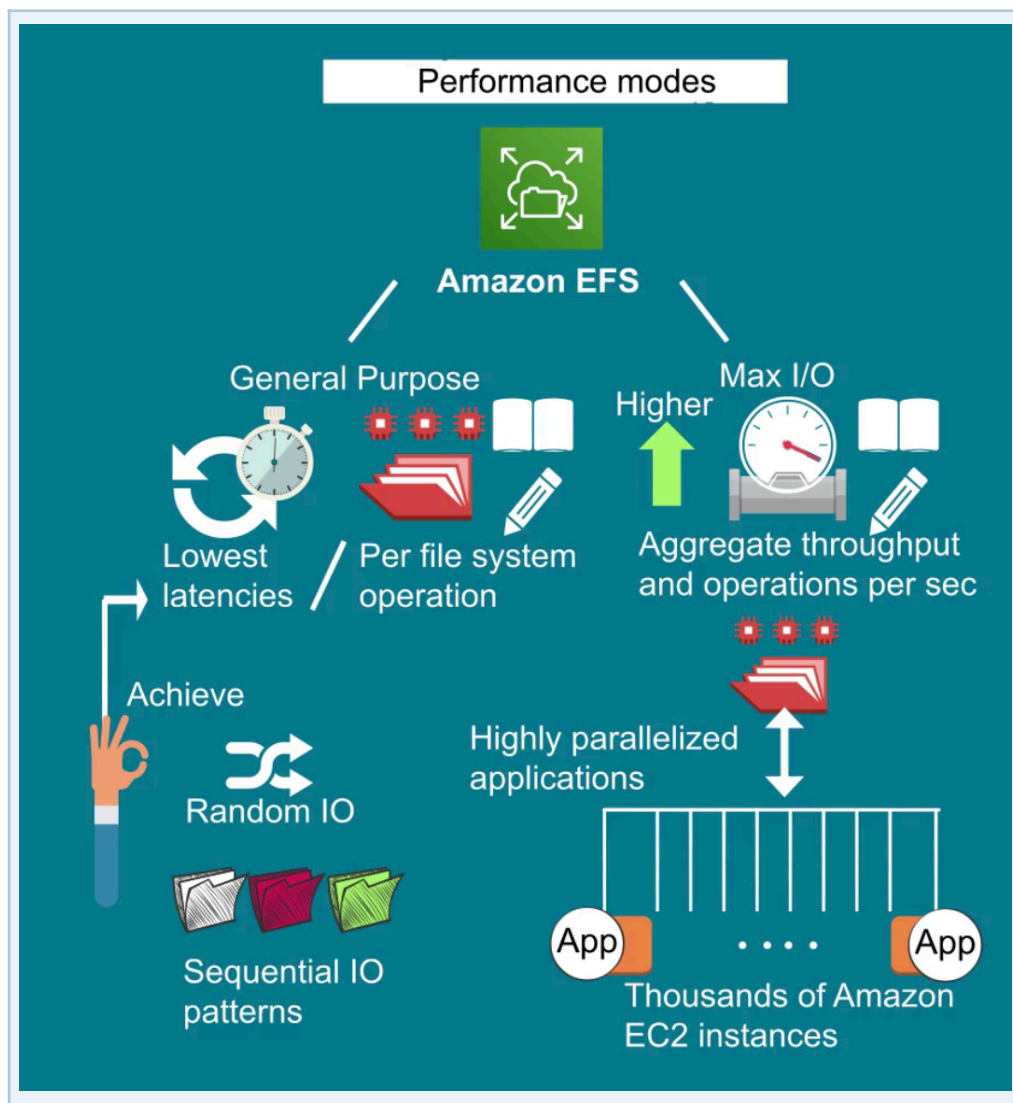
EFS tracks access and performs the transition automatically according to the policy.



Performance Modes

EFS performance modes determine whether a workload prioritises low latency or very large amounts of parallel activity.

Performance mode	Best for	Think of it as
General Purpose	Most applications that need fast file access	Faster individual operations
Max I/O	Very large workloads with many clients accessing EFS at once	More operations at the same time



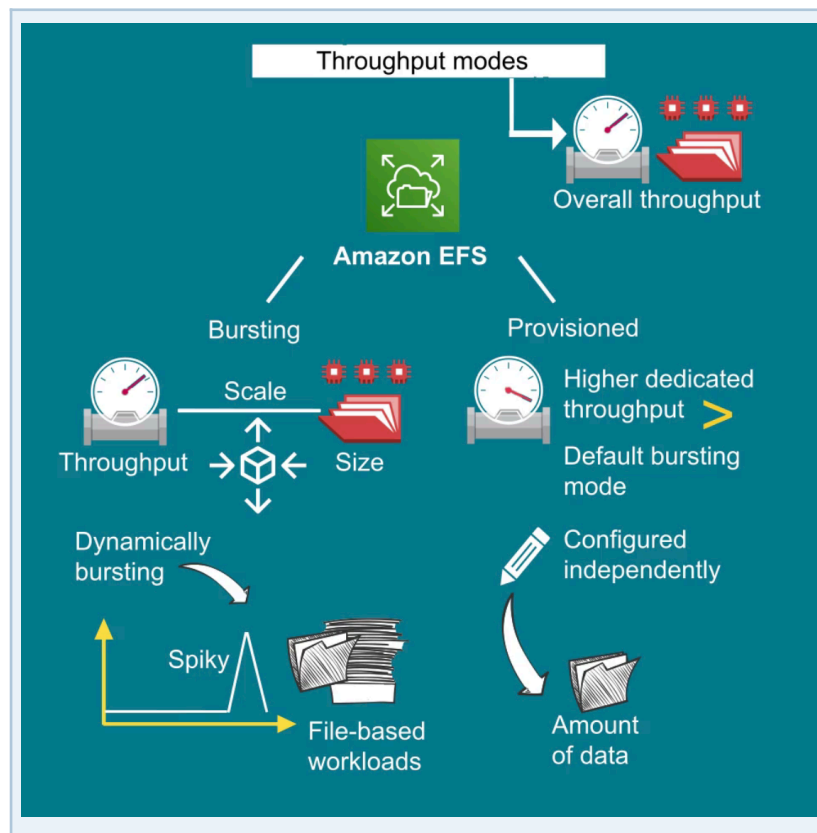
Throughput Modes

Throughput describes how much file data EFS can transfer over time. A workload can require high transfer rates even when the file system itself is relatively small.

Throughput mode	How it works	Best for
Bursting	Throughput scales with file-system size and can temporarily burst higher	Workloads with occasional spikes
Provisioned	Throughput is configured independently of stored data	Workloads needing consistently high throughput

Example: a photo-sharing app may normally have moderate activity but spike when many users upload at once. Bursting can handle these temporary peaks.

A video-processing service may continuously read and write large files throughout the day. Provisioned throughput can provide the consistently high transfer rate it requires.



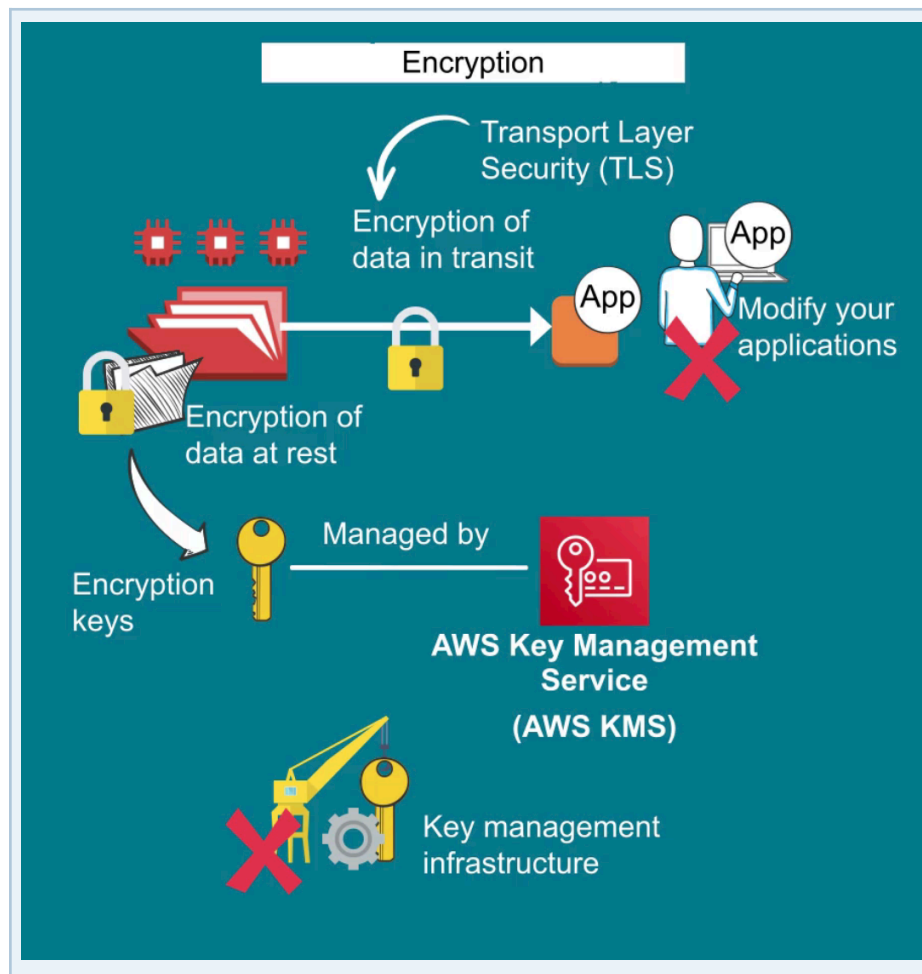
Summarised

Performance mode = HOW the filesystem handles the work. Throughput mode = HOW MUCH data can flow.

Encryption

Amazon EFS can protect files both while they are stored and while they travel over the network.

Situation	Protection
Data at rest	Files stored in EFS can be encrypted using keys managed through AWS KMS.
Data in transit	Communication between applications and EFS can be encrypted using TLS.

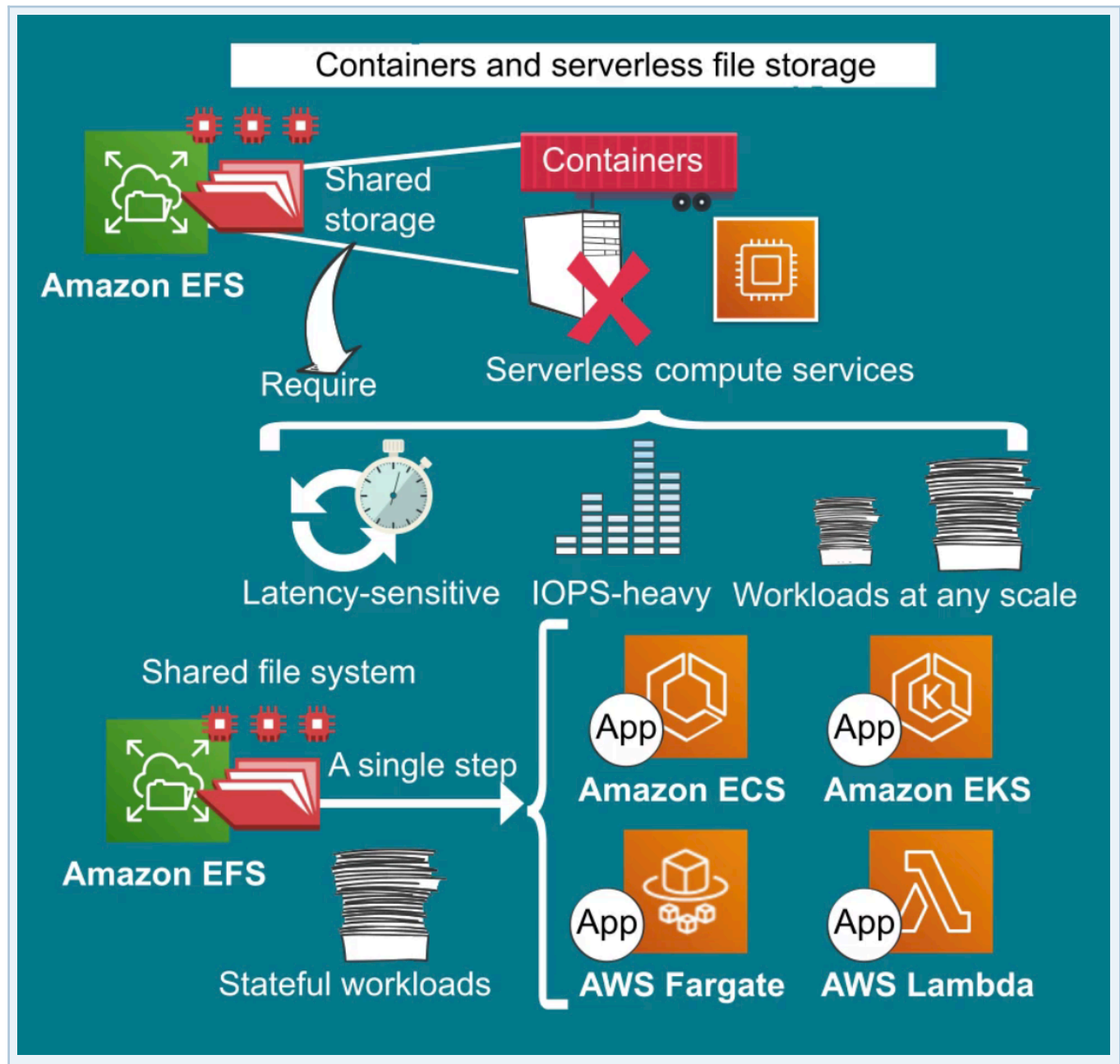


EFS encryption itself does not have a separate EFS encryption charge. Customer-managed KMS keys can introduce separate AWS KMS costs.

Containers and Serverless File Storage

Amazon EFS is not limited to normal EC2 instances. Shared EFS storage can also be used by container and serverless workloads, including Amazon ECS, Amazon EKS and AWS Lambda.

This is useful for stateful workloads because containers and serverless compute can be temporary. Important files can remain in EFS even when an application instance starts, stops or is replaced.



02 Practice Lab

In this practice lab we will:

- Launch and configure an Amazon EFS file system.
- Mount the file system to an Amazon EC2 instance.
- Connect a second EC2 instance to the same file system.
- Share files between the two EC2 instances.

Starting architecture

In this lab we have three separate web servers in separate Availability Zones.

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone
☐	WebServer1	i-0c7e3766f648ced98	Running	t3.micro	3/3 checks passec	us-east-1a
☐	WebServer2	i-0b8e117651d351711	Running	t3.micro	3/3 checks passec	us-east-1b
☐	WebServer3	i-0ef2813db981d7a28	Running	t3.micro	3/3 checks passec	us-east-1c

Reminder

Each EC2 instance must belong to at least one Security Group.

Security group name

webserver

webserver

webserver

Prepare the EFS Security Group

Before creating the EFS mount targets, we prepare a Security Group to control which resources are allowed to connect to EFS over NFS.

Setting	Value
Name	PetModels-EFS-1-SG
Description	Restrict access to web servers only
VPC	PetModels VPC
Inbound rule	NFS access from the web-server Security Group

Create security group [Info](#)

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)

Name cannot be edited after creation.

Description [Info](#)

VPC [Info](#)

Why an inbound rule?

The web servers need permission to connect to the EFS mount targets using NFS. Without an inbound rule, the EFS Security Group would block the connection.

Inbound rules [Info](#)

Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
NFS	TCP	2049	Custom	sg-0d77a7ec1f2f410e9	Delete
sg-0d77a7ec1f2f410e9					
Add rule					

After adding the inbound rule, create the Security Group.

Security group (sg-0044e2c642c6778bb | PetModels-EFS-1-SG) was created successfully

Details

sg-0044e2c642c6778bb - PetModels-EFS-1-SG

Actions

Details

Security group name

PetModels-EFS-1-SG

Security group ID

sg-0044e2c642c6778bb

Description

Restrict access to web servers only

VPC ID

vpc-0aeb711be65247ed3

Owner

408687975267

Inbound rules count

1 Permission entry

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (1)

Manage tags

Edit inbound rules

Search

	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-033aee082e588c0dc	-	NFS	TCP	2049

Looking for something different?

AWS also provides Amazon FSx, which offers managed file systems for Linux, Windows and macOS workloads. EFS is a general-purpose shared file system primarily for Linux workloads, while FSx is useful when a workload requires a specific filesystem technology, broader compatibility or specialised high-performance storage.

Amazon FSx

Launch and run feature-rich and highly performant file systems with just a few clicks

Get started

Create file system

How it works

Amazon FSx makes it easy and cost effective to launch and run popular file systems. With Amazon FSx, you can leverage the rich feature sets and fast performance of widely-used open source and commercially-licensed file systems, while avoiding time-consuming administrative tasks like hardware provisioning, software configuration, patching, and backups. It provides cost-efficient capacity and high levels of reliability, and it integrates with other AWS services so that you can manage and use the file systems in cloud-native ways.

Pricing

[Amazon FSx for NetApp ONTAP](#)

[Amazon FSx for OpenZFS](#)

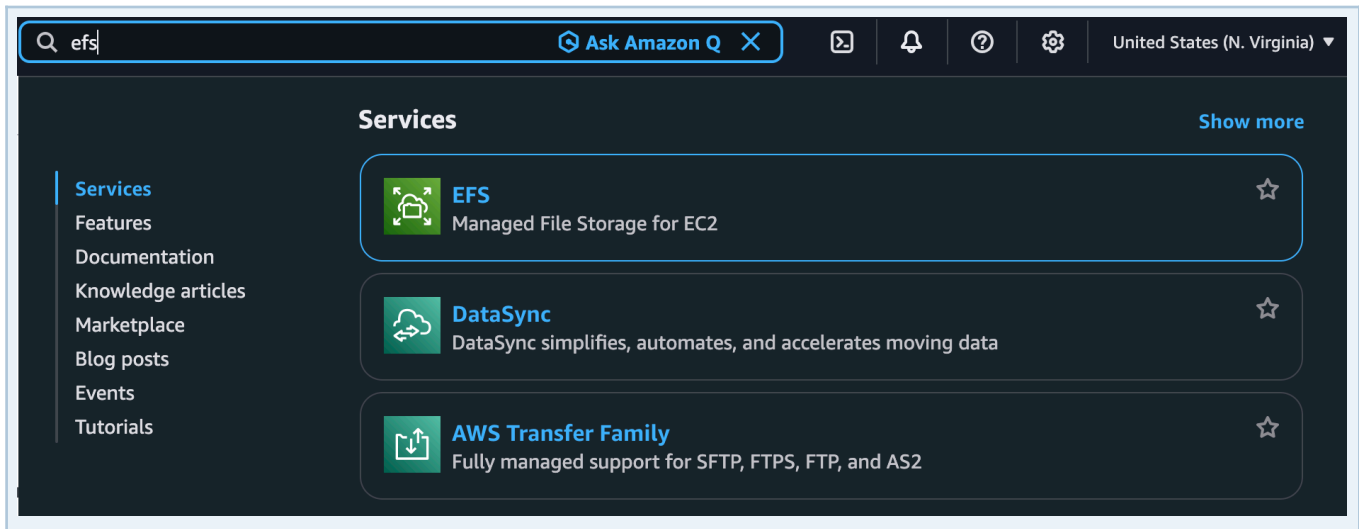
[Amazon FSx for Windows File Server](#)

[Amazon FSx for Lustre](#)

With Amazon FSx, you only pay for the resources you use.

Create the EFS File System

Amazon EFS provides a serverless elastic file system that lets applications share file data without provisioning or managing storage servers.



Choose Create file system.

EFS is not simply available to every AWS resource. Its mount targets are created inside the VPC you select.

Create file system

Create a file system with the recommended settings shown below by choosing Create file system. To view all settings or to customize your file system, choose Customise. [Learn more](#)

Name - optional
Name your file system.

PetModels-EFS-1

Name can include letters, numbers, and +-=_./ symbols, up to 256 characters.

Virtual Private Cloud (VPC)
Choose the VPC where you want EC2 instances to connect to your file system.

vpc-0aeb711be65247ed3
PetModels

Security Group recap

The web servers already have their own Security Group. We created a separate EFS Security Group that allows inbound NFS traffic on port 2049 only when it comes from those web servers. The rule takes effect when that Security Group is applied to the EFS mount targets.

Customise the File System

Recommended settings

Your file system is created with the following recommended settings unless you choose to customize the file system. You will be charged for storage and throughput. We recommend reviewing pricing for these features using the [AWS Pricing Calculator](#).

Setting	Value	Editable after creation
Throughput mode Learn more	Elastic	Yes
Transition into Infrequent Access (IA)	30 day(s) since last access	Yes
Transition into Archive	90 day(s) since last access	Yes
Transition into Standard	None	Yes

Cancel

Customize

Create file system

Amazon EFS can use Regional or One Zone storage.

- Regional stores data within and across multiple Availability Zones.
- One Zone stores data redundantly within a single Availability Zone at a lower price.

File system type

Choose to either store data across multiple Availability Zones or within a single Availability Zone. [Learn more](#)

☒ **Regional**
Offers the highest levels of availability and durability by storing file system data across multiple Availability Zones within an AWS Region.

☐ **One Zone**
Provides continuous availability to data within a single Availability Zone within an AWS Region.

For this temporary lab, automatic backup is disabled.

Setting	What happens
Standard	Normal storage for files that are regularly used
IA: 30 days	Files not accessed for 30 days move from Standard to lower-cost IA
Archive: 90 days	Files not accessed for 90 days move to lower-cost Archive

Setting	What happens
Return to Standard: None	Files moved to IA/Archive remain there even when accessed
Encryption	Stored files are encrypted using AWS KMS

Lifecycle management

Automatically save money as access patterns change by moving files into the Infrequent Access (IA) or Archive storage class. [Learn more](#)

Transition into Infrequent Access (IA)

Transition files to IA based on the time since they were last accessed in Standard storage.

30 day(s) since last access

Transition into Archive

Transition files to Archive based on the time since they were last accessed in Standard storage.

90 day(s) since last access

Transition into Standard

Transition files back to Standard storage based on when they are first accessed in IA or Archive storage.

None

Encryption

Choose to enable encryption of your file system's data at rest. Uses the AWS KMS service key (aws/elasticfilesystem) by default. [Learn more](#)

☒ Enable encryption of data at rest

► Customize encryption settings

In this lab, choose None for the IA, Archive and return-to-Standard transitions.

Before continuing, select Bursting.

Performance settings

Throughput mode

Choose a method for your file system's throughput limits. [Learn more](#)

☐ Enhanced

Provides more flexibility and higher throughput levels for workloads with a range of performance requirements.

☒ Bursting

Provides throughput that scales with the amount of storage for workloads with basic performance requirements.

► Additional settings

Configure Mount Targets

Keep the mount target for Web Server 1a, apply the PetModels EFS Security Group, and remove 1b and 1c for now. We will add another mount target later.

Mount targets

A mount target provides an NFSv4 endpoint at which you can mount an Amazon EFS file system. We recommend creating one mount target per Availability Zone. [Learn more](#)

Availability zone	Subnet ID	IP address type	IPv4 address	IPv6 address	Security groups	
us-east-... us-east-1a ✓ us-east-1b us-east-1c us-east-1d us-east-1e us-east-1f	subnet-...	IPv4 only	Optional	-	Choose ... sg-0044e2c642c6778bb PetModels-EFS-1-SG	Remove

CancelPreviousNext

Skip creating a file-system policy in this lab and continue.

Policy editor {JSON}

1

Manual changes will prevent the use of the policy options on the left until the editor is cleared.

CancelPreviousNext

Finalise the configuration and create the file system. Wait until its status becomes Available.

Attach EFS to the First EC2 Instance

The Attach option provides the command needed to mount EFS on an EC2 instance. Running the command connects a local folder to the EFS file system.

```
sudo mount -t efs -o tls fs-0f24efe2fd057c29d:/ efs
```

Attach ✕

Mount your Amazon EFS file system on a Linux instance. [Learn more](#)

☒ Mount via DNS ☐ Mount via IP

Using the EFS mount helper:

```
sudo mount -t efs -o tls fs-0f24efe2fd057c29d:/ efs
```

Using the NFS client:

```
sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsiz=1048576,hard,timeo=600,retrans=2,noresvport fs-0f24efe2fd057c29d.efs.us-east-1.amazonaws.com:/ efs
```

See our user guide for more information. [Learn more](#)

[Close](#)

Return to the EC2 instances and connect to the first web server using Session Manager.

Instances (1/3) [Info](#) Last updated less than a minute ago ↻ Connect [↗](#)

Saved filter sets
[Choose filter set ▼](#)

Instance state = running ✕ Clear filters ▼

☐	Name ✎ ▼	Instance ID	Instance state ▼	Instance type ▼
☒	WebServer1	i-0c7e3766f648ced98	✓ Running + -	t3.micro
☐	WebServer2	i-0b8e117651d351711	✓ Running + -	t3.micro
☐	WebServer3	i-0ef2813db981d7a28	✓ Running + -	t3.micro

Switch to the root user and install the EFS utilities. Administrative permissions are required to install the package and mount the file system.

```
sudo -i  
yum install -y amazon-efs-utils
```

```

sh-5.2$ sudo -i
[root@ip-10-10-0-209 ~]# yum install -y amazon-efs-utils
Last metadata expiration check: 1:23:50 ago on Mon Sep  7 11:34:32 2026.
Dependencies resolved.
=====
Package                                Architecture      Version           Repository        Size
=====
Installing:
amazon-efs-utils                       x86_64            3.3.1-1.amzn2023 amazonlinux        5.6 M
Installing dependencies:
stunnel                                x86_64            5.58-1.amzn2023.0.2 amazonlinux        156 k
Transaction Summary

```

Confirm that the utilities installed successfully.

Mount EFS and Create a Shared Log File

Create the local directory where EFS will appear:

```
mkdir data
```

Mount the EFS file system using TLS. Replace the final folder name from the copied command with data.

```

[root@ip-10-10-0-209 ~]# mkdir data
[root@ip-10-10-0-209 ~]# ls
data
[root@ip-10-10-0-209 ~]#

```

```
sudo mount -t efs -o tls fs-0f24efe2fd057c29d:/ data
```

```

[root@ip-10-10-0-209 ~]# mkdir data
[root@ip-10-10-0-209 ~]# ls
data
[root@ip-10-10-0-209 ~]#          sudo mount -t efs -o tls fs-0f24efe2fd057c29d:/ data

```

Move into the mounted directory:

```
cd data
```

Append text to a shared log file:

```
sudo bash -c "cat >> efs-1-setup.log"
```

Type:

efs-1 mounted in site A

Press CTRL+C to finish, then verify the file:

```
cat efs-1-setup.log
```

```
[root@ip-10-10-0-209 data]# sudo bash -c "cat >> efs-1-setup.log"
efs-1 mounted in site A
^C
[root@ip-10-10-0-209 data]# cat efs-1-setup.log
efs-1 mounted in site A
[root@ip-10-10-0-209 data]#
```

What just happened?

mkdir data created the local mount point. The mount command connected that folder to EFS. Writing efs-1-setup.log created the file in the shared EFS file system, and cat confirmed that it could be read back.

Add Another Mount Target

Return to File systems.

File systems (2)								
Filter by property values								
	Name ▾	File system ID ▾	Encrypted ▾	Total size ▾	Size in Standard ▾	Size in IA ▾	Size in Archive ▾	Provisioned Throughput (MiB/s)
	PetModels-EFS-1	fs-0f24efe2f-d057c29d	 Encrypted	12.00 KiB	12.00 KiB	0 Bytes	0 Bytes	-

Open Network and choose Manage.

Metered size	Monitoring	Tags	File system policy	Access points	Network	Replication
Network						 Manage

Add a mount target in Availability Zone 1b.

us-east-... ▲

subnet-... ▼

IPv4 only ▼

Optional

-

Choose ... ▼

Remove

us-east-1a

us-east-1b ✓

us-east-1c

us-east-1d

us-east-1e

us-east-1f

sg-0044e2c642c6778bb

X

PetModels-EFS-1-SG

Cancel

Save

Review the network configuration.

Network								
Availability zone (AZ-ID) ▲	Mount target ID ▼	Subnet ID ▼	VPC ID ▼	Mount target state ▼	IPv4 address ▼	IPv6 address ▼	Network interface ID ▼	Security groups ▼
us-east-1a (use1-az4)	fsmt-0fbb72c8aac9d631a	subnet-014b0fb6113bd4082	vpc-0aeb711be65247ed3	Available	10.10.0.93	N/A	eni-0c7ffca6b430e913b	sg-0044e2c642c6778bb (PetModels-EFS-1-SG)
us-east-1b (use1-az6)	fsmt-03626eecb1c3fe423	subnet-02c712678948bf7e9	vpc-0aeb711be65247ed3	Creating	10.10.1.211	N/A	eni-0b63fe6ce6db2f296	-

Connect the Second EC2 Instance

Return to EC2 and connect to Web Server 2 using Session Manager.

Instances (1/3) [Info](#)

Last updated less than a minute ago  [Connect](#) 

Saved filter sets
Choose filter set ▼

 Filter Instance by attribute or tag (case-sensitive)

Instance state = running 

Clear filters 

	Name 	▼	Instance ID	Instance state ▼	Instance type ▼
☐	WebServer1		i-0c7e3766f648ced98	 Running  	t3.micro
☒	WebServer2		i-0b8e117651d351711	 Running  	t3.micro
☐	WebServer3		i-0ef2813db981d7a28	 Running  	t3.micro

Repeat the same mounting steps:

```
sudo -i
yum install -y amazon-efs-utils
mkdir data
sudo mount -t efs -o tls fs-0f24efe2fd057c29d:/ data
cd data
sudo bash -c "cat >> efs-1-setup.log"
efs-1 mounted in site B
cat efs-1-setup.log
```

```
Complete!
[root@ip-10-10-1-12 ~]# mkdir data
[root@ip-10-10-1-12 ~]# sudo mount -t efs -o tls fs-0f24efe2fd057c29d:/ data
[root@ip-10-10-1-12 ~]# cd data
[root@ip-10-10-1-12 data]# sudo bash -c "cat >> efs-1-setup.log"
efs-1 mounted in site B
^C
[root@ip-10-10-1-12 data]# cat efs-1-setup.log
efs-1 mounted in site A
efs-1 mounted in site B
[root@ip-10-10-1-12 data]#
```

Wrapping Up

By the end of this lab, we have created a shared EFS file system that multiple EC2 web servers can access. Each server mounts the same EFS file system to its own local data folder through a mount target.

Because the files are stored centrally in EFS, the servers are not maintaining or synchronising separate copies. A file created or changed by one web server is available to the others using the same EFS file system.

Final mental model

EFS provides centralised, persistent and shared file storage that exists independently of the individual EC2 instances.